

BANCO DE DADOS – SQL 2

Profa. Vandecia Fernandes – BiCT/ECP/UFMA

Referências: Elmasri, R. and Navathe, S.B. Sistemas de Bancos de Dados. Korth, H.F. e Silberschatz, A. Sistemas de Bancos de Dados. Agradecimentos aos professores: Simara Rocha, Claudio Baptista e Eduardo Viana.



DML

DML

- **DML** (*Data Manipulation Language*): utilizada para o gerenciamento dos dados do banco.
- Operações básicas:
 - SELECT
 - INSERT
 - UPDATE
 - DELETE

DML

- Operações básicas:
 - SELECT- recuperar dados do banco de dados
 - INSERT – inserir dados em uma tabela
 - UPDATE – atualiza os dados existentes em uma tabela
 - DELETE – exclui registros de uma tabela



DML – INSERT

INSERT

- Usado para adicionar uma tupla a uma relação.

- Sintaxe:

```
INSERT INTO  <nome_tabela> [(<lista de atributos>)]  
VALUES (<lista de valores>);
```

INSERT

- Exemplos:

`INSERT INTO aluno (codigoA, nome) VALUES (20161, 'Joao da Silva');`

`INSERT INTO aluno VALUES (20161, 'Joao da Silva');`

- Podemos omitir o nome das colunas, desde que a ordem seja preservada;
- A inserção será rejeitada se tentarmos omitir um atributo que não permite valores nulos (NOT NULL)



DML – UPDATE

UPDATE

- Modifica o valor de atributos de uma ou mais tuplas.

- Sintaxe:

UPDATE tabela **SET** lista_atributos com atribuições de valores
[WHERE condição]

UPDATE

- Exemplos:

UPDATE aluno SET nome = 'novo'

UPDATE aluno SET nome = 'novo' WHERE codigoA = 1020

- Obs.: omitir a cláusula WHERE implica que o UPDATE deve ser aplicado a todas as tuplas da relação



DML – DELETE

DELETE

- Remove tuplas de uma relação

- Sintaxe:

DELETE FROM tabela [**WHERE** condição]

- Obs.: Se omitirmos a cláusula **WHERE**, então o **DELETE** deve ser aplicado a todas as tuplas da relação. A relação permanece no BD como uma relação vazia.

DELETE

- Exemplos:

DELETE FROM aluno

DELETE FROM aluno WHERE codigoA = 1020



DML – SELECT

SELECT

- Usado para recuperar informações do Banco de Dados;

- Sintaxe:

SELECT <lista atributos> **FROM** <lista tabelas> **WHERE** <condição>

- O resultado de qualquer comando **SELECT** é uma tabela
- O campo **FROM** realiza produto cartesiano das tabelas relacionadas
- **WHERE** deve trazer as condições de junção

SELECT

- SELECT
 - Lista os atributos desejados como resultados de uma consulta;
 - Corresponde a operação de projeção da álgebra relacional;
 - Lista de atributos pode ser substituído por “*” (todos);
- FROM
 - Lista de tabelas a serem usadas na execução;

SELECT

- WHERE
 - São definidos critérios de pesquisa envolvendo atributos das tabelas definidas na cláusula FROM;
 - Corresponde ao predicado da operação de seleção da álgebra relacional;
 - Comando opcional;
 - Comparação de valores : = , > , >= , < , <= , <> , or, and

SELECT

- Considere o esquema relacional abaixo para os exemplos a seguir:

EMP (cod_emp, nome , funcao, cod_gerente, data_admissao, salario, comissao, cod_depto)

FK cod_depto referencia Depto(cod_depto)

DEPTO (cod_depto, nome, localizacao)

SELECT

1. Selecionar todos os atributos de todos os empregados.
2. Selecionar o nome e código de todos os departamentos.
3. Selecionar todos os empregados com a função de Gerente.
4. Selecionar os nomes de todos os empregados Gerentes com salários maior que R\$ 5000
5. Mostrar o nome dos empregados e nome dos departamento onde trabalham.

SELECT

■ Respostas:

1. `SELECT * FROM emp;`
2. `SELECT cod_depto, nome FROM depto;`
3. `SELECT * FROM emp WHERE funcao = 'Gerente'`
4. `SELECT nome FROM emp WHERE funcao = 'Gerente' and salario > 5000;`
5. `SELECT emp.nome, depto.nome FROM emp, depto WHERE
emp.cod_depto=depto.cod_depto`

SELECT

- **ORDER BY**

- Especifica a sequência de ordenação dos resultados da consulta;
- Qualificador opcional : asc / desc.
 - **ASC**: crescente (padrão)
 - **DESC**: decrescente

- Sintaxe:

- `SELECT a1, a2,...,an FROM r1,r2,...rm WHERE P ORDER BY a1 DESC`

SELECT

- **ORDER BY** - exemplo:

- Selecionar para cada empregado o seu nome e a sua função ordenado primeiramente por função e depois por empregado (em ordem decrescente):

```
SELECT nome, funcao FROM emp ORDER BY funcao desc, nome  
desc
```

SELECT

- **BETWEEN**

- Faz uma pesquisa entre uma faixa de valores (**inclusive**) para um campo da tabela

- **NOT BETWEEN**

- Faz uma pesquisa descartando uma faixa de valores.

- Exemplo: Selecionar todos empregados que tenham comissão entre 0 e 1000:

```
SELECT * FROM emp WHERE comissao BETWEEN 0 AND 1000;
```

SELECT

- **IN**

- Consulta a presença de um campo em um conjunto de valores

- **NOT IN**

- Consulta a NÃO presença de um campo em um conjunto de valores

- Exemplo: Selecionar todos os empregados do departamento 5 e 6

```
SELECT * FROM emp WHERE cod_depto IN (5, 6)
```


SELECT

- **IS NULL**

- Verifica se o valor do campo comparado é vazio;

- **IS NOT NULL**

- Verifica se o valor do campo comparado não é vazio;

- Exemplo: Selecionar o nome, código e salário dos empregados que não possuem comissão.

```
SELECT cod_emp, nome ,salario FROM EMP  
WHERE comissao IS NULL;
```

SELECT

- **DISTINCT**

- Para eliminar as linhas duplicadas utiliza-se a palavra **DISTINCT** na cláusula **SELECT**.

- Exemplo:

```
SELECT DISTINCT cod_depto FROM EMP
```

SELECT

- **Cadeias de Caracteres (Strings):**
 - As strings devem aparecer entre **aspas simples**.
 - Os valores de caractere fazem distinção entre **maiúsculas** e **minúsculas**
- **Exemplo:**

```
SELECT nome, salario FROM emp WHERE nome = 'Jorge Sampaio'
```

SELECT

- **LIKE**

- Compara a existência de uma caractere em uma determinada posição em um string.
- ‘%’ representa qualquer sequência de N caracteres.

- **NOT LIKE**

- Compara a não existência de um caractere em uma determinada posição em um string.

SELECT

- **LIKE** - Exemplo:
- Selecionar o nome e código dos empregados cujo nome inicia com a letra 'C'.

```
SELECT cod_emp, nome FROM EMP  
WHERE nome LIKE 'C%';
```

SELECT

- Operadores Aritméticos
 - Adição: +
 - Subtração: -
 - Multiplicação: *
 - Divisão: /
- Exemplo: Selecionar o nome dos empregados, seu salário e o salário mais 300:

```
SELECT nome, salario, salario + 300 FROM emp;
```

SELECT

- Renomeando uma coluna ou tabela: **AS** ou somente o novo_nome

- Tabela:

SELECT * FROM empregado **AS** e OU

SELECT * FROM empregado e

- Coluna:

SELECT nome **AS** nome_empregado, salario **AS** sal FROM emp;

SELECT

- **CASE**

- Permite **mudar**** o valor de um dado

- **Exemplo**

```
SELECT mat, nome,  
CASE  
    WHEN sexo=1 THEN 'Masculino'  
    WHEN sexo=2 THEN 'Feminino'  
    WHEN sexo=0 THEN 'Indefinido'  
END, endereco, salario  
FROM Empregado
```


SELECT

- **Funções:**

- As funções são usadas para efetuar cálculos sobre dados, modificar itens individuais, manipular resultados de visualização e converter tipos de dados

SELECT – FUNÇÕES DE DATAS

- **Funções de Datas:**
 - A manipulação de datas é feita através de aspas simples
- **now():**
 - Retorna o timestamp do servidor;
 - `SELECT now();`
- **current_date:**
 - Retorna a data do servidor
 - `SELECT current_date;`

SELECT – FUNÇÕES DE DATAS

- **Funções de Datas:**
- **current_time():**
 - Retorna a hora do servidor
 - `SELECT current_time;`
- **date_part (retorna double)**
 - Retorna a parte especificada de uma data
 - `SELECT date_part('day', data_cadastro);`

SELECT – FUNÇÕES DE CARACTERES

- **Funções de manipulação de caracteres:**
- **LOWER(coluna):**
 - Converte as letras para minúsculas.
- **UPPER(coluna):**
 - Converte as letras para maiúsculas.

SELECT – FUNÇÕES DE CARACTERES

- **Funções de manipulação de caracteres:**

- Exemplo:

Select nome, funcao from Empregado where lower(funcao) = 'gerente'

SELECT – FUNÇÕES NUMÉRICAS

- **Funções numéricas:**
- **ROUND(valor):**
 - Arredonda o valor para o inteiro mais próximo
- **TRUNC(valor):**
 - Trunca o valor da coluna

SELECT – FUNÇÕES AGREGADAS

- Funções matemáticas que operam sobre as colunas
- Operações que são realizadas:
 - Soma = SUM;
 - Média = AVG;
 - Mínimo = MIN;
 - Máximo = MAX;
 - Contar = COUNT;

SELECT – FUNÇÕES AGREGADAS

- As funções de agrupamento podem ser utilizadas em conjunto com os comandos `DISTINCT` e `ALL`.
- `COUNT` pode ainda utilizar o `*`.

SELECT – FUNÇÕES AGREGADAS

- Exemplos:

- Selecionar a média de salário.

```
SELECT AVG(salario) FROM EMP;
```

- Selecionar total de salário pago pela empresa.

```
SELECT SUM(salario) FROM EMP;
```

SELECT – FUNÇÕES AGREGADAS

- Exemplos:

- Contar as tuplas da relação empregados.

```
SELECT COUNT(*) FROM EMP;
```

- Selecionar o maior salário entre os gerentes.

```
SELECT MAX(salario) FROM EMP WHERE funcao='Gerente'
```

SELECT – GROUP BY

- **GROUP BY**

- Forma grupos com as tuplas da tabela especificada na cláusula FROM, que possuem o mesmo valor no atributo especificado na cláusula *group by*;
- Para ter resultado em ordem, deve ser especificado também a cláusula *order by* (após a cláusula *group by*).

SELECT – GROUP BY

- **GROUP BY**

- Exemplo: Recuperar o número de empregados por departamento

- `SELECT cod_depto, COUNT(cod_emp) FROM EMP GROUP BY cod_depto`

SELECT – HAVING

- **HAVING**

- Usada em conjunto com GROUP BY para permitir a inclusão de condições nos grupos.
- Funciona parecido com a cláusula WHERE, mas:
 - WHERE: está relacionado a comparações envolvendo os valores de atributo de uma tupla;
 - HAVING: está relacionado a comparações envolvendo os resultados das funções agregadoras.

SELECT – HAVING

- **HAVING**

- Exemplo: Selecionar a média de salário dos departamentos com média superior a 700:

```
SELECT AVG(salario), cod_depto  
FROM EMP  
GROUP BY cod_depto  
HAVING AVG(salario)>700;
```

SELECT – JUNÇÃO

- Fazer uma junção corresponde a **selecionar dados de mais de uma tabela ao mesmo tempo;**
- Uma junção é feita utilizando uma condição ou um **conjunto** de condições;

SELECT – JUNÇÃO

- Resgata o relacionamento entre as tabelas: Comparação entre chave primária e chave estrangeira
 - Chave primária=chave estrangeira
- Cláusulas:
 - WHERE
 - JOIN

SELECT – JUNÇÃO

- WHERE

- Exemplo:

```
SELECT nemp, sal, local  
FROM emp, dep  
WHERE emp.ndep = dep.ndep;
```

SELECT – JUNÇÃO

- WHERE
- Caso as diferentes tabelas da junção possuam colunas com nomes iguais, é necessário distingui-los.
- Para fazer essa distinção, basta colocar o **nome da tabela.nome da coluna**
- **Exemplo:**
SELECT **e.nome**, sal, e.ndep, **d.nome**, local
FROM emp e, dep d
WHERE e.ndep = d.ndep;

SELECT – JUNÇÃO

- **JOIN**

- Combinação entre várias tabelas com base na relação existente entre elas.

- Variações:

- INNER JOIN;
- NATURAL JOIN;
- CROSS JOIN;
- OUTER JOIN;

SELECT – JUNÇÃO

- INNER JOIN:
- Junções internas ou *inner joins*, apresentam apenas os valores de ambas as tabelas que estão relacionados entre si.
- Uma junção interna implica que os dados que aparecem estejam relacionados.

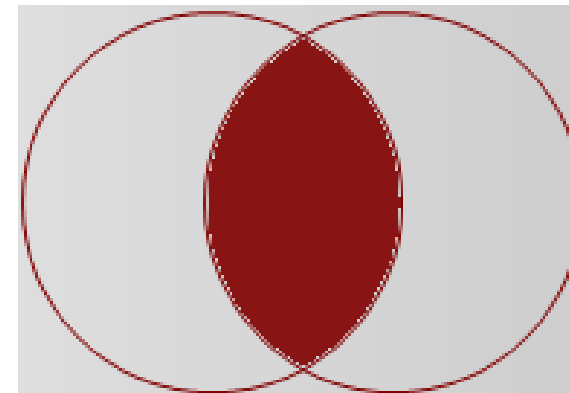
SELECT - JUNÇÃO

- INNER JOIN

Tabela1	
col	nome
1	João
2	José
3	Maria

Tabela2		
id	nome	col
1	Artes	2
2	Obras	3

col	nome	id	nome	col
2	José	1	Artes	2
3	Maria	2	Obras	3



```
SELECT *  
FROM Tabela1 A  
INNER JOIN Tabela2 B  
ON A.col = B.col
```

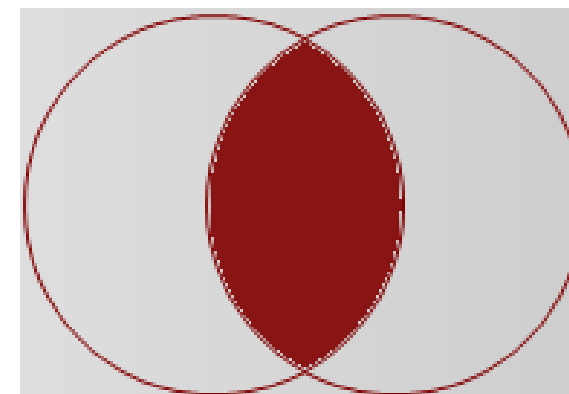
SELECT - JUNÇÃO

○ NATURAL JOIN

Tabela1	
col	nome
1	João
2	José
3	Maria

Tabela2		
id	nome	col
1	Artes	2
2	Obras	3

col	nome	id	nome	col
2	José	1	Artes	2
3	Maria	2	Obras	3



SELECT *
FROM Tabela1
NATURAL JOIN Tabela2 (col)

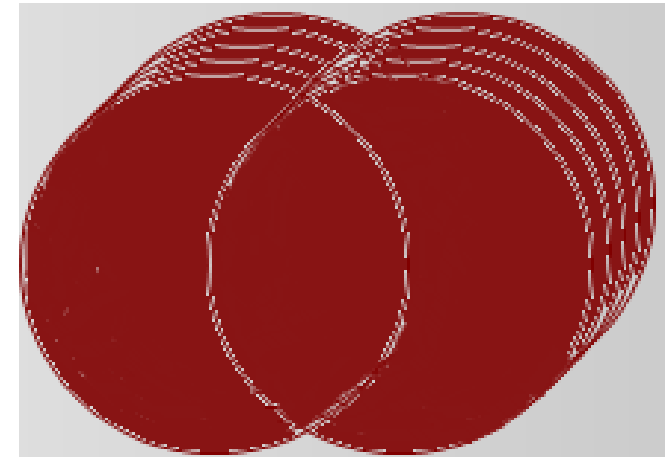
SELECT - JUNÇÃO

○ CROSS JOIN

Tabela1	
col	nome
1	João
2	José
3	Maria

Tabela2		
id	nome	col
1	Artes	2
2	Obras	3

col	nome	id	nome	col
1	João	1	Artes	2
1	João	2	Obras	3
2	José	1	Artes	2
2	José	2	Obras	3
3	Maria	1	Artes	2
3	Maria	2	Obras	3



```
SELECT *  
FROM Tabela1  
CROSS JOIN Tabela2
```

SELECT - JUNÇÃO

- OUTER JOIN:
- A junção externa ou *outer join* é utilizada quando for necessário mostrar, além dos dados da junção interna, todos os dados de uma tabela mesmo que não exista nenhum registro na outra tabela que se relacione com esse.

SELECT - JUNÇÃO

- OUTER JOIN:
- Variações:
 - LEFT OUTER JOIN
 - RIGHT OUTER JOIN
 - FULL OUTER JOIN

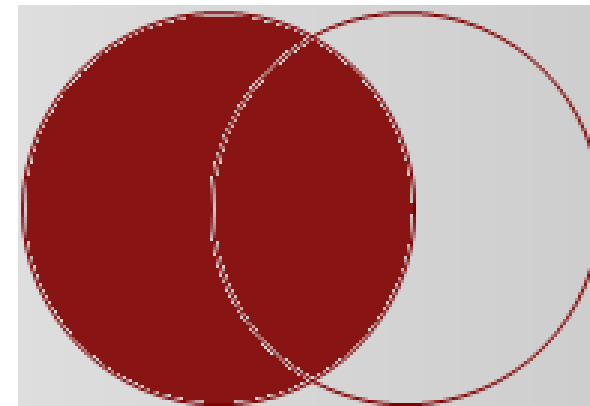
SELECT - JUNÇÃO

○ LEFT OUTER JOIN

Tabela1	
col	nome
1	João
2	José
3	Maria

Tabela2		
id	nome	col
1	Artes	2
2	Obras	3

col	nome	id	nome	col
1	João	NULL	NULL	NULL
2	José	1	Artes	2
3	Maria	2	Obras	3



```
SELECT *  
FROM Tabela1 A  
LEFT OUTER JOIN Tabela2 B ON  
A.col = B.col
```

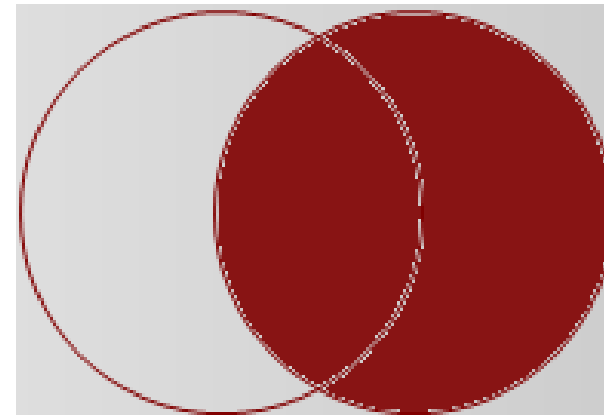
SELECT - JUNÇÃO

- RIGHT OUTER JOIN

Tabela1	
col	nome
1	João
2	José
3	Maria

Tabela2		
id	nome	col
1	Artes	2
2	Obras	3
3	Projetos	NULL

col	nome	id	nome	col
2	José	1	Artes	2
3	Maria	2	Obras	3
NULL	NULL	3	Projetos	NULL



```
SELECT *  
FROM Tabela1 A  
RIGHT OUTER JOIN Tabela2 B ON  
A.col = B.col
```

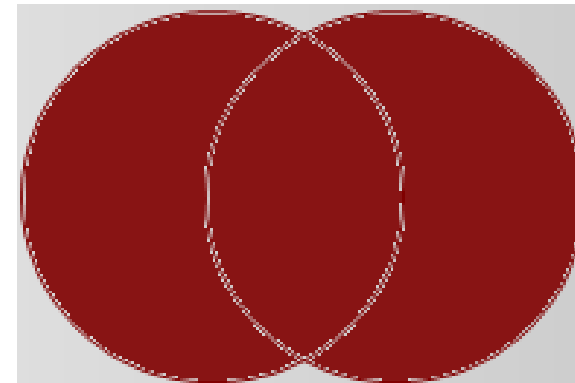
SELECT - JUNÇÃO

■ FULL OUTER JOIN

Tabela1	
col	nome
1	João
2	José
3	Maria

Tabela2		
id	nome	col
1	Artes	2
2	Obras	3

col	nome	id	nome	col
1	João	NULL	NULL	NULL
2	José	1	Artes	2
3	Maria	2	Obras	3
NULL	NULL	3	Projetos	NULL



```
SELECT *  
FROM Tabela1 A  
FULL OUTER JOIN Tabela2 B  
ON A.pk = B.fk
```

SELECT - JUNÇÃO

MARIA

Presente	Quantidade
Torradeira	1
Liquidificador	1
Panela	1

JOSÉ

Presente	Quantidade
Liquidificador	1
Televisão	1
Videogame	1

SELECT - JUNÇÃO

- INNER JOIN

- Mostra apenas os **presentes que estão nas duas listas** (em comum).

Presente	Quantidade Maria	Quantidade José
Liquidificador	1	1

- Somente “Liquidificador” aparece nas duas listas.

SELECT - JUNÇÃO

- LEFT JOIN

- Mostra **todos os presentes da Maria**, e indica se José também pediu o mesmo.

Presente	Quantidade Maria	Quantidade José
Torradeira	1	NULL
Liquidificador	1	1
Panela	1	NULL

- Todos os itens da Maria aparecem e onde José não pediu, fica vazio (NULL).

SELECT - JUNÇÃO

- RIGHT JOIN

- Mostra **todos os presentes do José**, e indica se Maria também pediu o mesmo

Presente	Quantidade Maria	Quantidade José
Liquidificador	NULL	1
Televisão	1	1
Videogame	NULL	1

- Todos os itens do José aparecem, e onde Maria não pediu, fica vazio.(NULL)

SELECT - JUNÇÃO

■ FULL JOIN

- Mostra **todos os presentes das duas listas**, mesmo que só um tenha pedido.
- É a união completa das listas — tudo que um dos dois pediu aparece.

Presente	Quantidade Maria	Quantidade José
Torradeira	1	NULL
Liquidificador	1	1
Panela	1	NULL
Televisão	NULL	1
Videogame	NULL	1

SELECT - JUNÇÃO

- CROSS JOIN
- Faz **todas as combinações possíveis** entre os presentes da Maria e os do José.
- *produto cartesiano* entre as duas tabelas.

Presente Maria	Presente José
Torradeira	Liquidificador
Torradeira	Televisão
Torradeira	Videogame
Liquidificador	Liquidificador
Liquidificador	Televisão
Liquidificador	Videogame
Panela	Liquidificador
Panela	Televisão
Panela	Videogame

SELECT – CONSULTAS ANINHADAS

- Algumas consultas precisam que os valores existentes no banco de dados sejam buscados e depois usados em uma condição de comparação.
- Para isso, utilizam-se consultas chamadas de subselect, que são como “selects dentro de selects”.
- Possuem consultas completas dentro da cláusula WHERE.

SELECT – CONSULTAS ANINHADAS

- Exemplo 1: Empregados que possuem salário maior do que a média:

```
SELECT nome, salario FROM EMP
```

```
WHERE salario > (SELECT AVG(salario) FROM EMP);
```

- Exemplo 2: Empregados que possuem salário menor do que a média dos gerentes com seu respectivo cargo:

```
SELECT nome, funcao, salario FROM EMP
```

```
WHERE salario < (SELECT AVG(salario) FROM EMP WHERE funcao =  
'gerente');
```

SELECT – CONSULTAS ANINHADAS

- Em consultas aninhadas podemos usar as cláusulas:
 - IN
 - SOME ou ANY
 - ALL
 - EXISTS.

SELECT – CONSULTAS ANINHADAS

- **IN** – visto no slide 24
- Exemplo: Selecionar o nome do empregado que possui o menor salário:

```
SELECT nome FROM emp WHERE salario IN (SELECT MIN(salario)  
FROM emp)
```

SELECT – CONSULTAS ANINHADAS

- **ANY ou SOME**

- Permite comparar um determinado valor com qualquer valor de um conjunto;
- Permite utilizar os operadores (>, >=, <, <=, =, <>)
- OBS.: A operação = SOME (ou = ANY) é equivalente a IN.
- OBS.: A operação <> SOME (ou <> ANY) é equivalente a NOT IN.

SELECT – CONSULTAS ANINHADAS

- **ALL**

- Permite comparar um determinado valor com todos os valores de um conjunto;
- Permite utilizar os operadores (>, >=, <, <=, =, <>);

- Exemplo:

```
SELECT nome, salario FROM emp  
WHERE salario >= ALL (SELECT salario FROM emp)
```


SELECT – CONSULTAS ANINHADAS

- **EXISTS ou NOT EXISTS:**
 - Usado para verificar se um determinado resultado é vazio ou não;
 - Se o resultado do consulta é:
 - Vazio, o EXISTS considerará o valor FALSE;
 - Não Vazio, o EXISTS considerará o valor TRUE;

SELECT – CONSULTAS ANINHADAS

- **EXISTS ou NOT EXISTS:**

- **Exemplo:**

```
SELECT e.nome FROM emp e
WHERE EXISTS (SELECT * FROM dependente d WHERE
e.cod_emp = d.cod_emp)
```

SELECT – OPERADORES DE CONJUNTOS

- **Operadores de Conjuntos:**
- Os operadores de conjuntos UNION, INTERSECT e MINUS permitem que se construam comandos com resultados de diferentes SELECTs combinados.
- Os comandos de SELECT a combinar podem referir-se a tabelas diferentes.

SELECT – OPERADORES DE CONJUNTOS

- **Operadores de Conjuntos:**
- UNIÃO (UNION)
- INTERSEÇÃO (INTERSECT)
- DIFERENÇA (MINUS ou EXCEPT)

SELECT – OPERADORES DE CONJUNTOS

- **Operadores de Conjuntos:**
- **Lembre-se:**
 - Mesmo grau
 - Domínios compatíveis
 - Cada uma dessas operações elimina automaticamente linhas duplicadas. Para mostrar todas as linhas use **ALL: UNION ALL, INTERSECT ALL e EXCEPT ALL.**

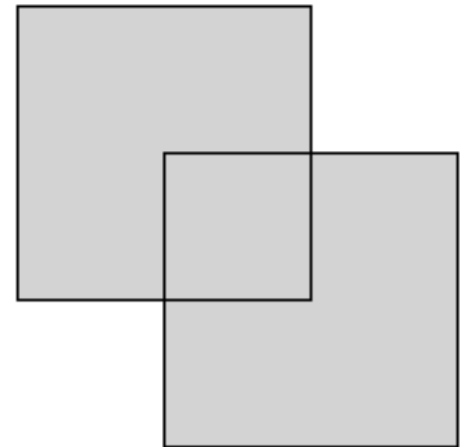
SELECT - UNIÃO

- UNIÃO (UNION)

- Exemplo:

```
SELECT funcao FROM emp WHERE ndep = 10  
UNION
```

```
SELECT funcao FROM emp WHERE ndep = 30  
ORDER BY funcao;
```

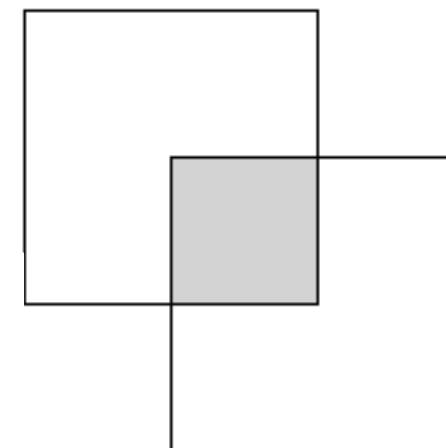


SELECT - INTERSEÇÃO

- INTERSEÇÃO (INTERSECT)

- EXEMPLO:

```
SELECT funcao FROM emp WHERE ndep = 10  
INTERSECT  
SELECT funcao FROM emp WHERE ndep = 30  
ORDER BY funcao;
```

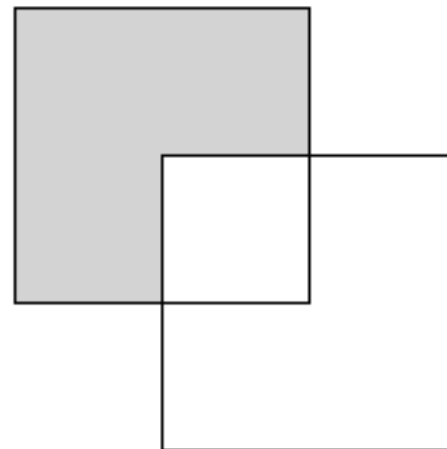


SELECT - DIFERENÇA

- DIFERENÇA (MINUS ou EXCEPT)

- EXEMPLO:

```
SELECT funcao FROM emp WHERE ndep = 10  
MINUS  
SELECT funcao FROM emp WHERE ndep = 30  
ORDER BY funcao;
```



SELECT - DIFERENÇA

- Exemplo:

- Recupere o nome dos empregados do departamento 5 que não são gerentes:

```
SELECT nome FROM emp WHERE cod_depto = 5
```

```
EXCEPT
```

```
SELECT nome FROM emp WHERE cod_depto = 5 AND  
funcao='Gerente'
```



DML – INSERT COM SELECT

INSERT COM SELECT

- **Sintaxe:**

```
INSERT INTO <nome_tabela> [(<lista de atributos>)]  
    <comando_select>;
```

- **Exemplo:**

```
INSERT INTO ENGENHARIA_COMP (cod_disciplina, nome)  
    SELECT cod, nome  
    FROM DISCIPLINAS  
    WHERE tipo=1;
```



DML – UPDATE COM SELECT

UPDATE COM SELECT

- Exemplo 1:

```
UPDATE produto SET preco=preco - 15 WHERE cod_produto IN  
(SELECT cod_produto FROM estoque)
```

- Exemplo 2:

```
UPDATE produto SET preco=preco - 15 FROM produto JOIN estoque  
ON produto.cod_produto=estoque.cod_produto
```



DML – DELETE COM SELECT

DELETE COM SELECT

- Exemplo:

```
DELETE FROM produto WHERE cod_produto NOT IN (SELECT  
cod_produto FROM estoque)
```



DÚVIDAS?